OPERAŢII PE BIŢI

```
c:
  0101110011110011
+ 0111000011110000
------------------
  1100110111100011
```

Adunare: 1 + 1 -> 0 şi carry 1

Scădere: 0 - 1 -> 1 şi carry -1

Înmulţire: Verific dacă se înmulţeşte cu o putere a lui 2 (2, 4, 8, 16).

Atunci shiftez la stânga (1, 2, 3, 4).

bit - **bi**nary digi**t**
baza x - are cifre de la 0 la x – 1

Numere pozitive:
Numărul Y din baza $X = \sum_0^{n-1} b_i \cdot X^i$, n este numărul de biţi din reprezentare
b_0 e LSB, b_{n-1} e MSB

Conversii:
- $B_{old} = 10$ are 10 cifre (0 – 9)
- $B_{new} = 100$ are 100 de cifre (0 – 99)

B -> B^p – Grupăm din B în câte P cifre, $p = \log_b B^P$

Numere negative (Pentru care, uneori, e nevoie de circuite speciale)
|S|_____ - S e MSB, bit rezervat pentru semn, deci sunt disponibili n – 1 biţi din reprezentare!

Numărul -Y din baza $X = -b_i \cdot 2^{n-1} + \sum_0^{n-2} b_i \cdot 2^i$

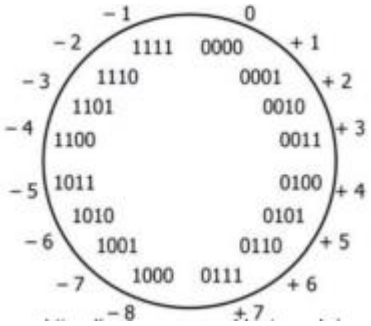
Complement faţă de doi:

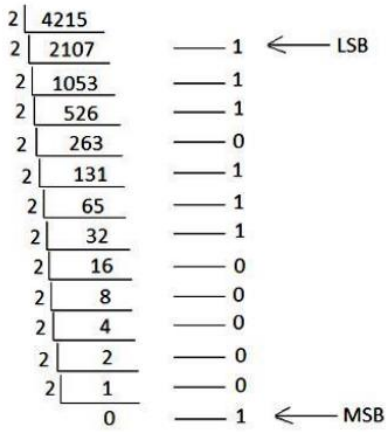
$(-x)_{b10}$ -> b2 1. Scriem |x| în binar 2. Inversăm biţii 3. Adăugăm 1

$(-x)_{b2}$ -> b10 1. Scriem numărul aşa cum e 2. Inversăm biţii 3. Adăugăm 1

Folosim sistemul binar: Acelaşi algoritm de adunare, pot fi folosite aceleaşi circuite pentru adunare naturală

- **un exemplu explicit: $(4215)_{10} = (1000001110111)_2$**





Cel mai mare număr care se poate reprezenta pe N biți: $2^N - 1$

Cel mai mare număr care se poate reprezenta (complement față de 2) pe N biți: $2^{N-1}-1$
Cel mai mic: -2^{N-1}

X – Număr natural. Biți necesari pentru reprezentare: ceil($\log_2 X$)
X – K cifre în HEX, de câți biți e nevoie pentru BIN: K * $\log_2 16$ = 4 HEX_{old} BIN_{new}
X – K cifre în BIN, de câți biți e nevoie pentru HEX: ceil(k / 4) BIN_{old} HEX_{new}
X – K cifre în DEC, de câți biți e nevoie pentru BIN: ceil(k * $\log_2 10$) DEC_{old} BIN_{new}

De ce funcționează complementul față de 2?

$$-\left(-2^N + \sum_{i=0}^{N-1} b_i 2^i\right) = 2^N - \sum_{i=0}^{N-1} b_i 2^i$$

$$2^{N+1} = \sum_{i=0}^{N} 2^i + 1$$

$$= \sum_{i=0}^{N-1} 2^i + 1 - \sum_{i=0}^{N-1} b_i 2^i$$

$$= \sum_{i=0}^{N-1} (1 - b_i) 2^i + 1$$

$$= (\text{inversam bitii}) + 1$$

BINARY FIXED-POINT

...	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	2^{-1}	2^{-2}	2^{-3}	2^{-4}	2^{-5}	2^{-6}	2^{-7}	...

2^-1	0.5
2^-2	0.25
2^-3	0.125
2^-4	0.0625
2^-5	0.03125
2^-6	0.015625
2^-7	0.0078125
2^-8	0.00390625
2^-9	0.001953125
2^-10	0.0009765625
2^-11	0.00048828125
2^-12	0.000244140625
2^-13	0.0001220703125

Ex: 101.<u>101</u>

$$\begin{cases} 101 = 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 4 + 1 = 5 \\ 101 = 1 \cdot 2^{-1} + 0 \cdot 2^{-2} + 1 \cdot 2^{-3} = 0.5 + 0.125 = 0.625 \end{cases}$$

⇨ $101.\underline{101} = 5.625$
⇨

Ex: 3.<u>75</u>

$$\begin{cases} 3 = 1 \cdot 2^1 + 1 \cdot 2^0 \\ 0.75 = 1 \cdot 2^{-1} + 1 \cdot 2^{-2} \end{cases}$$

⇨ $3.\underline{75} = 11.11$

LOGARITM ÎNTREG

9. Demonstrați că $\lfloor \log_2 x \rfloor = i_{\max}$ unde x este un număr dat pe N biți iar $i_{\max} = \max\{i \mid b_i = 1,\ \forall\, i = 0, \ldots, N-1\}$ unde b_i reprezintă al i-lea bit din reprezentarea binară a numărului x. De exemplu: dacă $x = 00101110$ (46 zecimal) atunci $\lfloor \log_2 x \rfloor = 5$.

- **arătați că** $\lfloor \log_2 x \rfloor = i_{\max}$
- **pornim de la reprezentarea binară și aplicăm logaritmul**

$$x = \sum_{i=0}^{N-1} b_i 2^i$$

$$\begin{aligned}
\log_2 x &= \log_2 \left(\sum_{i=0}^{N-1} b_i 2^i \right) \\
&= \log_2 \left(2^{i_{\max}} \left(\sum_{i=0}^{N-1} b_i \frac{2^i}{2^{i_{\max}}} \right) \right) \\
&= \log_2 2^{i_{\max}} + \log_2 \left(\left(\sum_{i=0}^{N-1} b_i \frac{2^i}{2^{i_{\max}}} \right) \right) \\
&= i_{\max} + C,\ C < 1
\end{aligned}$$

OVERFLOW-SAFE BINARY-SEARCH

```
int binarySearch1(int arr[], int start, int end, int x)
{
        if (end >= start)
        {
                int mid = start + (end - start) / 2;

                if (arr[mid] == x)
                        return mid;

                if (arr[mid] > x)
                        return binarySearch1(arr, start, mid - 1, x);

                return binarySearch1(arr, mid + 1, end, x);
        }

        return -1;
}
```

GENERAL / ISTORIC

ENIAC – 1945 – 1955 / USA – 1000 op/s (aka Electronic Numerical Integrator and Computer)
HPE CRAY – 2021 / USA – 1714 petaoperații/s
”compute” – Infrastructura hardware pe care rulează algoritmii
Putem scădea costul de calcul pentru ML prin: algoritmi eficienți, hardware dedicat

Varianta A – Cea mai rapidă
(Calculul a două matrice)

```
# varianta A
for (int i = 0; i < n; ++i)
  for (int j = 0; j < n; ++j)
     for (int k = 0; k < n; ++k)
        C[i][j] += A[i][k] * B[k][j];
```

Complexitate: $O(n^3)$
Operații elementare: Adunare, înmulțire
Numărul operațiilor elementare: 2 pentru fiecare for (pe lângă asta, în fiecare for sunt câte n operații elementare de incrementare)
Număr de operații: $2 * n^3$

PERSONALITĂȚI

Richard Hamming - “The purpose of computing is insight, not numbers.”. Codul Hamming pentru integritatea mesajelor transmise la distanțe mari.

Blaise Pascal – Creează Pascaline (1642) – Calculator mecanic, capabil de +-, jucăria arisotcrației | Limbajul Pascal e numit în onoarea lui

Gottfried Wilhelm Von Leibniz – Studiază sistemul binar, extinde mașina lui Pascal (adaugă * /)

George Boole – Scrie ”The Laws of Thought” | Introduce logica booleană și analizează operațille logice de bază: Negație / Conjucție / Disjuncție / Disjuncție exclusivă | Ele stau la baza teoriei informației

Charles Babbage – Proiectează Mașina Diferențială Nr. 2 – prima mașină de calcul mecanică programabilă – prototipuri de 13 tone – Tatăl calculatoarelor moderne

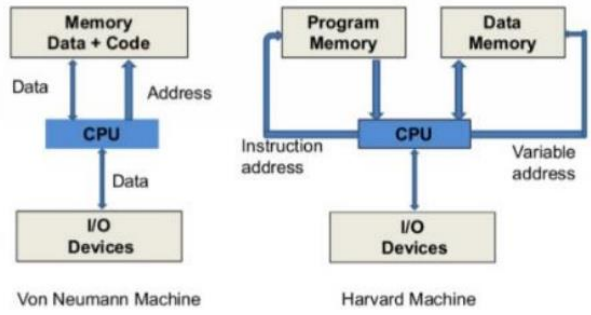
Ada Lovelace – Colaborează cu Babbage | Scrie un program care calculează numerele numere Bernoulli – Nu existau limbaje de programare, dar scrie pași pentru execuție – Primul programator

Konrad Zuse – Introduce o serie de calculatoare: Z1, Z2, Z3, Z4 | Folosește sistemul binar | Instrucțiuni stocate pe o bandă | Introduce reprezentarea și calculul în virgulă mobilă / floating point | Aproape totul în izolare, pe perioada celui de-Al II-lea Război Mondial.

Alan Turing – Decriptează rapid mesaje Enigma folosind mașina The Bomb - Brute-Force search | Introduce Mașina Turing pentru algoritmi Turing Complete – Sistem de analiză și recunoaștere a unui număr mare de date | Introduce Testul Turing – Pot mașinile să gândească?

John Von Neumann – Contribuie la crearea primului calculator electronic: ENIAC | Creează EDVAC – Sistem binar cu programe stocate. Introduce arhitectura von Neumann

Arhitectura von Neumann:



Claude Shannon: Părintele teoriei informației (o inventează literally) | Demonstrează că problemele de logică booleană se pot rezolva cu circuite electronice | Teorema de eșantionare Shannon – Nyquist: Analog <-> Digital fără a pierde date

Grace Hopper – Contribuitoare în dezvoltarea de limbaje high-level și scrie compilatorul COBOL

Margaret Hamilton - Scrie cod pentru software-ul de la bord pentru misiunea Apollo

Barbara Liskov – Design pattern: Principiul substituției Liskov + altele în distributed computing

Rivest Shamir Adleman – Creatorii primului sistem de criptare cu chei, public: RSA. Foloește numere prime. Foarte util în tranzacțiii bancare.

Diffie Hellman – Schimbul de cheie Diffie-Hellman – Soluția în a trimite mesaje secrete într-un canal de comunicație nesecurizat.

Ritchie and Thompson – Dezvoltatorii limbajului C | Pun bazele sistemelor open-source, creează Unix OS.

Richard Stallman – Contributori la GNU Project.

Linus Tolvads – Creatorul Linux și GIT (pentru controlul versiunilor)

Steve Jobs – Cofondează Apple, Pixar

Bill Gates – Fondator Microsoft <3 | Acum CEO este Satya Nadella

Jeff Bezos – Fondator & fost CEO Amazon

Mark Zuckerberg – CEO Meta

Larry Page and Sergey Brin – Fondatorii Google

Idei mari:

- De la mecanic la electric
- De la o maşină care face un singur lucru la o maşină programabilă
- Design modular (pe module)
- Teorie despre ce este posibil pe aceste maşini
- Dorinţa de a face lucrurile optim, la limită şi fără risipă

După Al Doilea Război Mondial, dorinţa de cercetare în domeniul calculatoarelor creşte în ritm exponenţial

Actori importanţi: Grupuri profesionale (IEEE, ACM, Bell Labs) şi state (Statele Unite: DARPA)

TEORIA PROBABILITĂȚILOR

$$P = \frac{\text{Numărul cazurilor favorabile}}{\text{Numărul cazurilor posibile}}$$ (atunci când evenimentul nu este influențat de mediul exterior)

$P1 \cdot P2$ (probabilitatea ca două evenimente **INDEPENDENTE** să se întâmple, fără să fie influențate de mediul exterior)Este intersecția dintre evenimentul care se putea întâmpla primul și cele totale. Trebuie să ținem cont de cazurile care sunt favorabile pentru ambele evenimente.

TEORIA INFORMAȚIEI

Informația: Date care afectează (aproape mereu reduce) incertitudinea despre un fenomen.
Se poate acumula, este constantă (nu este creată/distrusă).

$I(x_i) = \log_2\left(\frac{1}{p_i}\right)$ (Câtă informație ne oferă o valoare în funcție de probabilitatea ca ea să apară)

Calculăm cantitatea de informație folosind X (variabila aleatoare):
N – Numărul de valori distincte: $x_1, x_2 \dots x_n$
P - Fiecare valoare apare cu probabilitatea $p_1, p_2 \dots p_n$
p_i mai mic -> Obținem cantitate mai mare de informație

Putem privi formula astfel: $I(x_i) = \log_2\left(\frac{1}{M\frac{1}{N}}\right) = \log_2\left(\frac{N}{M}\right) =$ rezultat în biți!

N – Numărul total de evenimente
M – Numărul de evenimente favorabile

- **entropia**
  - valoarea medie de informației primită despre o variabilă X

$$H(X) = E(I(X)) = \sum_{i=1}^{N} p_i \log_2 \frac{1}{p_i} = \sum_{i=1}^{N} -p_i \log_2 p_i$$

  - *H(X)* se numește entropia lui X
  - *I(X)* este informația despre X
  - *E* este "expected value", operația care calculează valoarea medie

  - exemplu: X = {A, B, C, D} cu probabilități {1/3, ½, 1/12, 1/12}
    - $H(X) = -\frac{1}{3}\log_2\frac{1}{3} - \frac{1}{2}\log_2\frac{1}{2} - \frac{1}{12}\log_2\frac{1}{12} - \frac{1}{12}\log_2\frac{1}{12} = 1.626$ biti

I) 12 piese: 3 cuburi, 3 conuri, 3 sfere, 3 piramide | Fiecare formă are una dintre culorile: roşu, galben, albastru

A extrage o piesă roşie. Câtă informaţie primeşte copilul B, care vrea să afle tipul piesei?

Inţial: 12 variante, dar 4 piese sunt de culoare roşie, deci $p_i = 4 \cdot \frac{1}{12}$

$$I(x_i) = \log_2\left(\frac{1}{4 \cdot \frac{1}{12}}\right) = \log_2(3)$$ (x_i este o piesă roşie)

II) În urnă: 5 bile roşii, 3 bile albastre

Se extrage o bilă albastră.
Primim I(bilă albastră) cantitate de informaţie

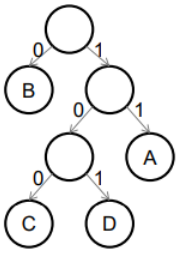
$$I(\text{bila albastra}) = \log_2\left(\frac{1}{\frac{3}{8}}\right) = \log_2\left(\frac{8}{3}\right) = 1.42 \text{ biti}$$

$$H(\text{urna}) = \frac{5}{8}\log_2\left(\frac{8}{5}\right) + \frac{3}{8}\log_2\left(\frac{8}{3}\right) = 0.95 \text{ biti}$$

$$H(\text{urna dupa extragere}) = \frac{5}{7}\log_2\left(\frac{7}{5}\right) + \frac{2}{7}\log_2\left(\frac{7}{2}\right) = 0.86 \text{ biti}$$

Entropia spune că: Nu se poate face o reprezentare pe biţi mai optimă decât valoarea ei fără a pierde informaţie. (Ceva compresat perfect se numeşte zgomot)
Este limita de compresie posibilă!

Codarea eficientă şi unică se face cu arbori binari:
Frunzele – Codurile
Stânga/Dreapta – Decis de 0/1



Implementare: Algoritmul lui Huffman
(Optim când considerăm un singur caracter pe rând)
Input: Probabilitatea fiecărui eveniment, ex: {1/3, 1/2, 1/12, 1/12}
Output: Codurile de pe un arbore binar
Cheia: O codare mai scurtă pentru event. cu probabilităţi mari, una mai scurtă -- //-- mici.
Dacă toate evenimentele sunt echiprobabile, nu putem face nimic.

```
00100111010011   0 0 100 11 101 0 0 11
                 B B C   A  D   B B A (Parcurgem arborele)
```

Detectarea erorilor: Ineficientă ca stocare.

Metoda I) Adăugăm biţi de paritate simbolurilor, pentru a recupera informaţia.

Corecţia erorilor: Distanţă Hamming – O distanţă mare poate duce şi la corectarea erorilor
O distanţă Hamming de 2N+1 poate corecta E erori. Distanţa hamming minimă pentru 4 biţi schimbaţi este 4.

Orice comunicaţie / Stocare este redundantă!

Aproximarea lui Stirling: $\log_2(a!) \approx a\log_2(a) - a\log_2 e$

Probabilităţile pentru fiecare caz:

(În principiu : Cât ştim / Cât vrem să ştim)

a) x are exact două valori "1" în reprezentarea sa binară: $p = \frac{C_N^2}{2^N}$, pp. n par.

b) x are exact N/2 valori "1" în reprezentarea sa binară (N par): $p = \frac{C_N^{N/2}}{2^N}$, pp. n par.

c) x are o secvenţă continuă de N/4 biţi de "1" (restul sunt "0", N divizibil la 4) :

$p = \frac{(3N/4+1)}{2^N}$, pp. 4 | n

d) x are MSB (Most Significant Bit) setat la "1": $p = \frac{2^{N-1}}{2^N} = \frac{1}{2}$

e) x este impar: $p = \frac{2^{N-1}}{2^N} = \frac{1}{2}$

f) x este o putere a lui 2: $p = \frac{N}{2^N}$

g) x are primii N/2 biţi din reprezentarea sa binară setaţi la "0" (N par): $p = \frac{2^{N/2}}{2^N}$

h) x este un număr prim (aproximativ estimat):

$$p \approx [2N/l\,n(2N)]2N = 1l\,n(2N)\,p \approx \frac{[2^N/\ln(2^N)]}{2^N} = \frac{1}{\ln(2^N)}$$

i) x are un număr par de biţi setaţi la "1" (N par): $p = \frac{1+\sum_{i=1}^{N/2} C_N^{2i}}{2^N} = \frac{1}{2}$

j) x = 42: $p = \frac{1}{2^N}$

ASSEMBLY x86

Utilizări: Securitate informatică: Hacking, Reverse Engineering | Optimizare: Dezvoltare jocuri şi ML / AI | Debugging | Dezvoltare software low-level: Sisteme embedded / Sisteme de operare

În Assembly x86, rezultatul înmulţirii poate fi stocat pe 64 de biţi (%edx * 2^{32} + %eax)

Shift: |S|XXX.XXXX >> 2 = 00|S|X.XXXX – Se pierde bitul de semn!
SAR/SAL – Shift artimetic, ţine cont de semn.

jmp *%eax – Sare la o adresă

aritate 1 – Operaţia se aplică unui singur număr

.asciz -> Lungimea şirului + 1 pentru \n

S1: ”ASC”, S2: ”FMI” -> syscall de print pentru $S1 cu lungimea 5 -> ASCFM

¬ NOT (are aritate 1) ∧ AND ∨ OR Tsunotshi DX – eu irl | ORNOTAND

CMP – Compară două valori şi setează flag-uri:
- Zero Flag (ZF) op1 = op2
- Sign Flag (SF) Rezultatul comparaţiei este negativ
- Carry Flag (CF) op1 < op2

a(b, c, d) = a + b + c · d

operand	Descriere	Semn (Da/Nu)	Flaguri setate	Condiţie Flag
jc	jump dacă este carry setat	Nu	CF (Carry Flag)	CF = 1
jnc	jump dacă nu este carry setat	Nu	CF	CF = 0
jo	jump dacă este overflow setat	Nu	OF (Overflow Flag)	OF = 1
jno	jump dacă nu este overflow setat	Nu	OF	OF = 0
jz	jump dacă este zero setat	Nu	ZF (Zero Flag)	ZF = 1
jnz	jump dacă nu este zero setat	Nu	ZF	ZF = 0
js	jump dacă este sign setat	Da	SF (Sign Flag)	SF = 1
jns	jump dacă nu este sign setat	Da	SF	SF = 0
jb	jump if below (unsigned $op2 < op1$)	Nu	CF	CF = 1
jbe	jump if below or equal (unsigned $op2 \leq op1$)	Nu	CF, ZF	CF = 1 or ZF = 1
ja	jump if above (unsigned $op2 > op1$)	Nu	CF, ZF	CF = 0 and ZF = 0
jae	jump if above or equal (unsigned $op2 \geq op1$)	Nu	CF	CF = 0
jl	jump if less than (signed $op2 < op1$)	Da	SF, OF	SF ≠ OF
jle	jump if less than or equal (signed $op2 \leq op1$)	Da	SF, OF, ZF	SF ≠ OF or ZF = 1
jg	jump if greater than (signed $op2 > op1$)	Da	SF, OF, ZF	SF = OF and ZF = 0
jge	jump if greater than or equal (signed $op2 \geq op1$)	Da	SF, OF	SF = OF
je	jump if equal ($op1 = op2$)	Nu	ZF	ZF = 1
jne	jump if not equal ($op1 \neq op2$)	Nu	ZF	ZF = 0

TABELA 1. Operandi de salt condiţionat, flagurile setate şi condiţiile lor

CIRCUITE

Circuit digital combinațional

La ieșire, este o combinație (funcție logică care combină) toate sau o parte a intrărilor

Nu are stări interne
- Pui un semnal digital constant la intrare
și ai un semnal digital constant la ieșire
- Nu poate memora nimic
- Nu are o stare internă (memorie)

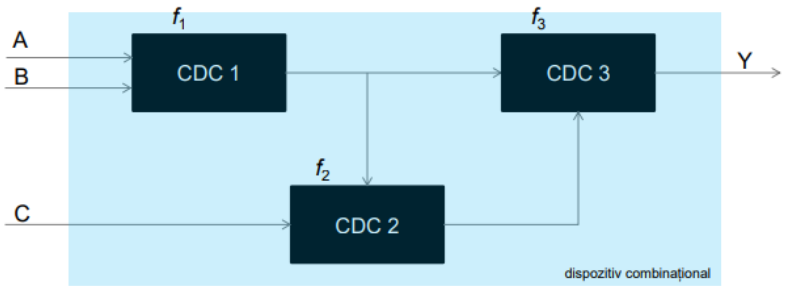
Timp de propagare t_0: Timp maxim necesare pentru a produce la ieșire semnale digitale corecte și valide

Pentru fiecare intrare, trebuie să știm care e ieșirea

A	B	C	X	Y
0	0	0	0	1
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	0

Dispozitiv combinațional

Elementele sale: Circuite combinaționale
O intrare este conectată la exact o ieșire / la o constantă
Fără cicluri în graful direcțional al dispozitivului



Funcția dispozitivului: Y = f3(f1(A, B), f2(f1(A, B), C)) – Ne uităm ce intră în Y șamd.

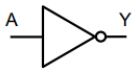
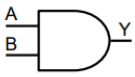
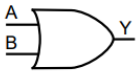
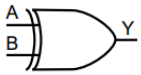
Timpul total de propagare: $t_{p,\,total} = t_{p,\,1} + t_{p,\,2} + t_{p,\,3}$ (cea mai lungă cale)
Timpul maxim după care avem o ieșire validă dacă avem intrări valide

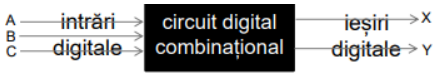
Un computer care funcționează la 1GHz trimite comenzi o dată la 1ns

De ce folosim semnale digitale în loc de analogice:
- Într-un sistem analogic zgomotul se acumulează
- Într-un sistem digital, avem corecțiile de zgomot (avem margini)

De ce folosim sistemul binar?
- Număr redus de stări (2) (pentru HEX ar trebui 16, deci ar trebui distinse 16 nivele de voltaj)
- Pentru baza 4, am avea 4 nivele, deci am fi de 2 ori mai eficienți

Tot ce facem pe sistemul de calcul trebuie redus la circuite care sunt porți logice

Operația	Valori	Notație
NOT		A	Y 0	1 1	0	$\bar{A}$ (A bar, A complement)
AND		A B Y 0 0 0 0 1 0 1 0 0 1 1 1	AB
OR		A B Y 0 0 0 0 1 1 1 0 1 1 1 1	A + B
XOR	 A B Y 0 0 0 0 1 1 1 0 1 1 1 0 $A \oplus B = \bar{A}B + A\bar{B}$	$A \oplus B$



A	B	C	X	Y
0	0	0	0	1
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	0

o expresie booleană care conține regulile din tabel?

- $X = \bar{A}\bar{B}C + \bar{A}B\bar{C} + A\bar{B}\bar{C} + ABC$

poartă	întârziere (ps)	suprafață (μm²)
AND-2	50	25
NAND-2	30	15
OR-2	55	26
NOR-2	35	16
AND-4	90	40
NAND-4	70	30
OR-4	100	42
NOR-4	80	32

N* - Mai rapide, mai mici ca suprafață față de *
*-2 – Mai rapide, mai mici ca suprafață față de *-4

Tranzitoare CMOS: Circuitele analogice sunt mai eficiente pe logică negată (mai puține componente electrice)

CIRCUIT	INPUT	OUTPUT	NR INTRĂRI	NR IEȘIRI
ADUNARE	A (N biți) B (N biți)	S (N + 1 biți)	2N	N + 1

Cât de mare va fi circuitul? $(N + 1)2^{2N-1}$